OpenCode + OpenRouter Agentic Setup Guide

This document provides a practical setup and operating model for using OpenCode with OpenRouter, Kimi, and Claude as a production workflow for agentic coding, SRE/infrastructure work, trading automation, and AI product development.[cite:1] [cite:2] [cite:3] [cite:24] [cite:25] [cite:26] [cite:30]

Objective

The recommended stack is a tiered model: use OpenCode as the execution shell, OpenRouter as the routing layer, Kimi K2.6 as the default builder model, Kimi K2 Thinking as the lower-cost planning model, and Claude only as a premium escalation layer for final reviews or high-risk design checks.[cite:26] [cite:30] [cite:7] [cite:8] [cite:16]

This design is intended to reduce waiting on vendor-specific usage caps, lower average token cost, preserve long-context capability, and improve day-to-day throughput on multi-step projects.[cite:7] [cite:8] [cite:13] [cite:16] [cite:19]

Why this setup

OpenCode supports CLI-based model selection, session continuation, session forking, file attachment, and agent selection, which makes it well suited for parallel, multi-session workflows instead of one long monolithic thread.[cite:26]

OpenRouter supports provider routing behavior, including preferred provider ordering and automatic fallback or load balancing, which helps keep work moving when one provider path becomes slow or unavailable.[cite:30] [cite:35] [cite:37]

OpenRouter currently lists Kimi K2.6 with a 262,144-token context window at $0.95 per million input tokens and $4 per million output tokens, while Kimi K2 Thinking is listed as a lower-cost long-horizon reasoning option with a similar large context window.[cite:7] [cite:8] [cite:31]

Coverage of Claude Code rate limits shows rolling and weekly constraints for heavy continuous usage, which explains why using Claude as the default worker can become a productivity bottleneck during sustained coding sessions.[cite:13] [cite:16] [cite:19]

Core architecture

Layer	Recommended choice	Role
Execution shell	OpenCode [cite:26][cite:27]	Terminal-native coding agent environment
Routing gateway	OpenRouter [cite:30][cite:35]	Provider abstraction, routing, fallback
Planner model	Kimi K2 Thinking [cite:8][cite:31]	Decomposition, reasoning, milestone planning
Builder model	Kimi K2.6 [cite:7]	Code generation, repo-wide edits, implementation

Layer	Recommended choice	Role
Premium reviewer	Claude [cite:16] [cite:19]	Final review, architecture audit, high-risk checks

Recommended routing policy

The workflow should route tasks by task class rather than forcing one model to handle everything.[cite:7] [cite:8] [cite:16]

Task class	Primary model	Notes
Planning, decomposition, requirements digestion	Kimi K2 Thinking [cite:8] [cite:31]	Use before implementation to keep cost lower
Implementation, refactors, tests, docs	Kimi K2.6 [cite:7]	Default daily workhorse
Final code review, blast-radius analysis, architecture judgment	Claude [cite:16] [cite:19]	Use selectively for premium judgment
Outage fallback	Alternate provider path via OpenRouter [cite:30] [cite:37]	Prevent workflow freezes

A practical policy is to let Kimi handle most daily loops and reserve Claude for the small fraction of tasks where the extra judgment materially changes the outcome.[cite:7] [cite:16] [cite:19]

OpenCode setup

OpenCode documentation and setup guides show support for model selection using `--model`, session continuation using `--continue` or `--session`, session forking using `--fork`, and file attachment for working from repo briefs or design documents.[cite:26] [cite:12]

A recommended operating pattern is to keep three active sessions per project:

- Planner session: converts objective and repo state into milestones and risks.[cite:26] [cite:8]
- Builder session: implements one milestone at a time using the compressed project brief. [cite:26] [cite:7]
- Reviewer session: audits correctness, tests, docs, rollback safety, and cleanup.[cite:26] [cite:16]

Environment pattern

OpenCode setup material for OpenRouter shows using OpenRouter as the OpenAI-compatible API base, with environment variables such as `OPENAI_API_BASE=https://openrouter.ai/api/v1` and an OpenRouter API key.[cite:12] [cite:36]

A practical shell pattern for macOS or Linux is:

```
export OPENAI_API_BASE="https://openrouter.ai/api/v1"
```

```
export OPENAI_API_KEY="<your_openrouter_api_key>"
```

Session examples

The following CLI patterns align with documented OpenCode capabilities for model choice, file attachment, continuation, and forking.[cite:26]

```
opencode --model openrouter/moonshotai/kimi-k2-thinking
opencode --model openrouter/moonshotai/kimi-k2.6-20260420
opencode --continue
opencode --continue --fork
opencode --file project-brief.md --file repo-map.md
```

OpenRouter provider strategy

OpenRouter documentation describes intelligent routing behavior and provider selection, including the ability to specify a preferred provider order and let the platform fall back when necessary.[cite:30] [cite:35] [cite:37]

The recommended policy is:

- Use default routing or load balancing for general development work.[cite:30]
- Use explicit provider order when stability or repeatability matters more.[cite:30]
- Keep fallbacks enabled for most development tasks so work does not pause during provider degradation.[cite:30] [cite:37]
- Disable fallback only on tasks where reproducibility is more important than continuity.[cite:30]

Daily operating workflow

Morning spin-up

Start each project or milestone with a planning session in Kimi K2 Thinking using the objective, repo summary, constraints, and acceptance criteria.[cite:8] [cite:31]

Ask for:

- Problem framing
- Milestone plan
- File-level change plan
- Risks and assumptions
- Test strategy
- Questions requiring human confirmation

This creates a reusable working brief without immediately consuming premium review capacity.[cite:8] [cite:31]

Build loop

Run implementation in a separate Kimi K2.6 session using only the current milestone, the compressed project brief, and the relevant files.[cite:7] [cite:26]

This keeps context focused while still allowing large-context work when repo-wide awareness is genuinely needed.[cite:7]

Review loop

Fork a review session once the milestone is stable, and use either Kimi K2.6 for routine review or Claude for release-critical review and tricky architecture decisions.[cite:26] [cite:7] [cite:16] [cite:19]

End-of-day compression

At the end of each session, generate a compressed handoff summary containing current objective, completed work, changed files, unresolved risks, next actions, and assumptions to preserve.[cite:7] [cite:8]

This compressed state becomes the next session's seed and is one of the highest-leverage methods for reducing repeated token spend.[cite:7] [cite:8]

Prompt templates

Planner template

Use in Kimi K2 Thinking.[cite:8] [cite:31]

```
You are my project planner.

Goal:
[one-sentence project outcome]

Context:
[product/system context]

Constraints:
- stack:
- runtime:
- security/compliance:
- performance:
- deadline:

Inputs:
- repo summary:
- relevant files:
- current blockers:

Output exactly:
1. problem framing
2. milestone plan
```

```
3. file-level change plan
4. risks and assumptions
5. test strategy
6. questions that require my answer
```

Builder template

Use in Kimi K2.6.[cite:7]

```
You are my implementation agent.

Objective:
[one milestone only]

Working rules:
- change only files needed
- preserve existing patterns
- avoid unnecessary rewrites
- explain tradeoffs briefly
- if uncertain, propose smallest safe implementation first

Project summary:
[paste compressed brief]

Relevant files:
[list]

Deliver:
1. exact edits
2. tests added/updated
3. rollback risks
4. concise handoff note
```

Reviewer template

Use in Kimi K2.6 for routine review or Claude for premium review.[cite:7] [cite:16] [cite:19]

```
Act as a strict reviewer.

Review for:
- correctness
- edge cases
- performance
- maintainability
- security
- observability
- missing tests

Context:
[compressed brief + diff summary]

Return:
1. critical issues
```

```
2. medium issues
3. nice-to-have improvements
4. approve / do not approve
```

Session compressor template

Use at the end of every session to preserve reusable state and reduce transcript replay.[cite:7] [cite:8]

```
Compress this session for future continuation.

Return:
- current objective
- completed work
- files changed
- unresolved risks
- next 3 actions
- assumptions to preserve
- discardable context
```

Token optimization rules

The most important cost-saving move is prompt discipline rather than relying only on a cheaper model.[cite:7] [cite:8]

Use these operating rules:

- Do not attach the full repo by default; start with a repo map and targeted files.[cite:7] [cite:26]
- Do not continue from long raw history if a compressed brief exists.[cite:7] [cite:8]
- Keep one milestone per implementation session.[cite:7]
- Use session forking for review or alternate implementations instead of bloating one thread.[cite:26]
- Pass summaries, diffs, and manifests instead of re-sending full files whenever possible.[cite:7] [cite:8]
- Maintain persistent repo guidance files for stable context.[cite:26]

Recommended repo support files

For recurring SRE, infrastructure, trading, and agentic projects, each repository should include a small operational context layer.[cite:1] [cite:2] [cite:4] [cite:23]

- `project-brief.md`: project goal, architecture notes, constraints, coding rules.
- `repo-map.md`: important folders, core files, dangerous paths, ownership notes.
- `runbook.md`: run, test, deploy, rollback procedures.
- `handoff.md`: compressed latest working state for the next session.

These files are especially useful for high-context systems such as infra automation, trading pipelines, and multi-agent products where continuity matters as much as code generation speed.[cite:2] [cite:4] [cite:23]

Project-specific application

SRE and infrastructure

Use Kimi K2 Thinking for planning, Kimi K2.6 for implementation, and Claude for deploy-sensitive changes, permissions, auth, rollback logic, or anything with wider operational blast radius.[cite:8] [cite:7] [cite:16] [cite:19]

Trading and research automation

Use Kimi K2 Thinking for strategy decomposition and research workflows, Kimi K2.6 for scripts and adapters, and Claude only for capital-sensitive execution logic or critical safety checks. [cite:2] [cite:23] [cite:8] [cite:7] [cite:16]

Agentic product work

Use Kimi K2 Thinking for decomposition, Kimi K2.6 for building and iteration, and Claude for launch-level judgment, architecture review, or hard product trade-offs.[cite:1] [cite:25] [cite:8] [cite:7] [cite:16]

Cost guardrails

A practical operating model is to define three usage lanes.[cite:7] [cite:8] [cite:16]

Lane	Model	Rule
Cheap planning lane	Kimi K2 Thinking [cite:8] [cite:31]	Use first for decomposition and summarization
Main build lane	Kimi K2.6 [cite:7]	Use by default for implementation work
Premium review lane	Claude [cite:16] [cite:19]	Use only after build completion or when blast radius is high

Additional guardrails:

- Do not use Claude for first drafts.[cite:16] [cite:19]
- Do not use Claude for exploratory repo scanning.[cite:16]
- Do not use Claude for repetitive test scaffolding.[cite:16]
- Use Claude only when the extra judgment is worth the higher cost or lower throughput. [cite:16] [cite:19]

Minimal command discipline

OpenCode setup guides describe repo-native command patterns such as `/init`, `/plan`, `/build`, and `/improve`.[cite:12]

A disciplined routine is:

- `/init` once per repo to establish context.[cite:12]
- `/plan` at the start of each milestone.[cite:12]
- `/build` for implementation.[cite:12]
- `/improve` only after the milestone is mostly stable.[cite:12]
- `--continue` for same-track continuation and `--fork` for review or alternative paths.[cite:26]

This keeps planning, implementation, and review from collapsing into one overgrown context window.[cite:12] [cite:26]

Final recommendation

The most effective long-term setup is to standardize on OpenCode as the primary coding surface, OpenRouter as the gateway layer, Kimi K2 Thinking as the planning model, Kimi K2.6 as the implementation model, and Claude as a scarce premium reviewer.[cite:26] [cite:30] [cite:8] [cite:7] [cite:16]

This combination is well aligned with a workflow centered on agentic productivity, SRE and infrastructure work, trading automation, and AI product experimentation, while reducing the cost and interruption patterns caused by heavy dependence on Claude alone.[cite:1] [cite:2] [cite:3] [cite:24] [cite:25] [cite:16]